KAKEIBO APP

設計書

生成元: docs/design.md | 生成日時: 2026/04/24 23:37

設計書

1. システム概要

本システムは、家計の記録、分析、予算管理、AI 支援を 1 つの Web アプリで提供する Next.js ベースの家計簿です。フロントエンドと API を同一リポジトリの App Router で管理し、データは Supabase、認証は Auth0、配信は Vercel を利用します。

2. 全体構成

```
Browser
  ├─ App Router UI
  ├─ Dashboard / Input / Settings / Contact
  └─ Fetch to /api/*

Next.js App Router
  ├─ src/app/*                画面
  ├─ src/app/api/*            API Route Handlers
  ├─ src/lib/components/*     UI コンポーネント
  ├─ src/lib/auth/*           認証連携
  ├─ src/lib/server/*         セキュリティ / サーバーユーティ
リティ
  └─ src/lib/supabase/*       DB アクセス

External Services
  ├─ Supabase                 profiles / transactions /
budgets
  ├─ Auth0                    Google / LINE / メール認証
  ├─ OpenAI / Gemini 等       AI 機能
  ├─ Google Maps API          周辺店舗検索 / Geocoding（優
先）
  ├─ Nominatim (OSM)          エリアジオコーディング（フォール
バック）
  ├─ Overpass API (OSM)       周辺店舗検索（フォールバック）
  ├─ Resend                   メール送信
  └─ Vercel                   Hosting / Build / Deploy
```

3. レイヤ設計

3.1 Presentation Layer

- `src/app/`
- `src/lib/components/`

役割:

- 家計入力画面、ダッシュボード、設定、法務ページ、問い合わせページを表示
- API 呼び出し結果を反映
- 一部 UI 状態をローカルストレージに保持

3.2 Application Layer

- `src/app/api/`

役割:

- 認証済みユーザーのリクエストを受けて入力検証を実行
- same-origin、rate limit、認可を通した上で業務処理を実行
- 外部 API と DB の橋渡しを行う

3.3 Domain / Utility Layer

- `src/lib/utils.ts`
- `src/lib/fixed-cost-flags.ts`
- `src/lib/components/*` 内の計算ロジック

役割:

- 家計サマリー、予算差分、固定費判定、生活支援ロジックの計算

3.4 Infrastructure Layer

- `src/lib/supabase/admin.ts`
- `src/lib/auth0.ts`
- `src/lib/server/security.ts`
- `src/lib/server/ssrf.ts`

役割:

- Supabase 管理クライアント

- Auth0 セッション処理
- JSON 読み取り制限、same-origin、rate limit
- SSRF 対策

4. データ設計

主要テーブル:

- `profiles`
- `transactions`
- `budgets`

補助データ:

- 予算差額
- 予算トレードオフ
- 引き落とし予約

基本方針:

- ユーザー単位でデータを分離
- Supabase の RLS を前提にする
- サーバー側では Auth0 セッションから解決した `supabaseUserId` を利用して書き込み対象を固定する

5. 認証設計

認証方式:

- Auth0 セッション認証
- Google OAuth
- LINE Login
- メール系ログイン / 確認フロー

設計ポイント:

- 画面系は Auth0 セッション前提
- API はセッション確認と same-origin を併用
- webhook 系 API は same-origin ではなく署名検証で保護

6. セキュリティ設計

実装済みの主要制御:

- `requireSameOrigin` による CSRF 軽減
- `rateLimit` による濫用抑制
- `readJsonBody` によるサイズ制限つき JSON 読み取り
- `boundedText` による入力長制限
- webhook の JWT / 署名検証
- SSRF 対策ユーティリティ

今回の補強:

- `src/app/api/profile/route.ts` に same-origin と rate limit を追加

7. AI 店案内設計

対象:

- `src/lib/components/NearbyShopGuide.tsx`
- `src/app/api/nearby-shops/route.ts`

処理フロー:

1. 画面で店種、エリア、自由入力キーワードを指定
2. エリア入力時: `/api/nearby-shops` に送信 → Google Geocoding API（失敗時は Nominatim）でジオコーディング
3. 現在地使用時: ブラウザの Geolocation API で座標を取得 → `/api/nearby-shops` に送信
4. サーバーで Google Places API（New）を呼び出し（`GOOGLE_MAPS_API_KEY` 設定時）
5. Google が失敗または 0 件の場合、サーバーから Overpass API（OSM）でフォールバック検索
6. 距離順に最大 6 件へ整形し表示
7. 0 件時はエラーコードと検索座標を診断情報としてレスポンスに付与し UI に表示

設計意図:

- サーバーが認証・レート制限・ジオコーディング・店舗検索をすべて担当
- Google Maps API が有効な場合は優先して使用し、失敗または 0 件時は OSM へ自動フォールバック
- OSM（Nominatim / Overpass）は無料・APIキー不要で常に予備として機能

- Overpass 障害時は予備エンドポイント（overpass.kumi.systems）へ自動切り替え
- 0 件時は Google/OSM のエラーコードと検索座標を診断情報として返し、原因の特定を容易にする

8. デプロイ設計

環境:

- ローカル開発
- Vercel Preview
- Vercel Production

本番 URL: `https://kawaii0214.vercel.app`

デプロイ方針:

- Git push → main ブランチ → Vercel 自動デプロイ
- 本番環境変数は Vercel 管理
- `.env.local` はローカル専用

9. 非機能要件

- TypeScript による型安全
- ESLint による静的検証
- Next.js build 通過をリリース基準にする
- セキュリティ境界を API 側で強制する

10. 今後の改善候補

- API ごとの rate limit 方針を明文化
- 監査ログの追加
- README と docs の多言語整理
- AI 店案内の監視指標追加